



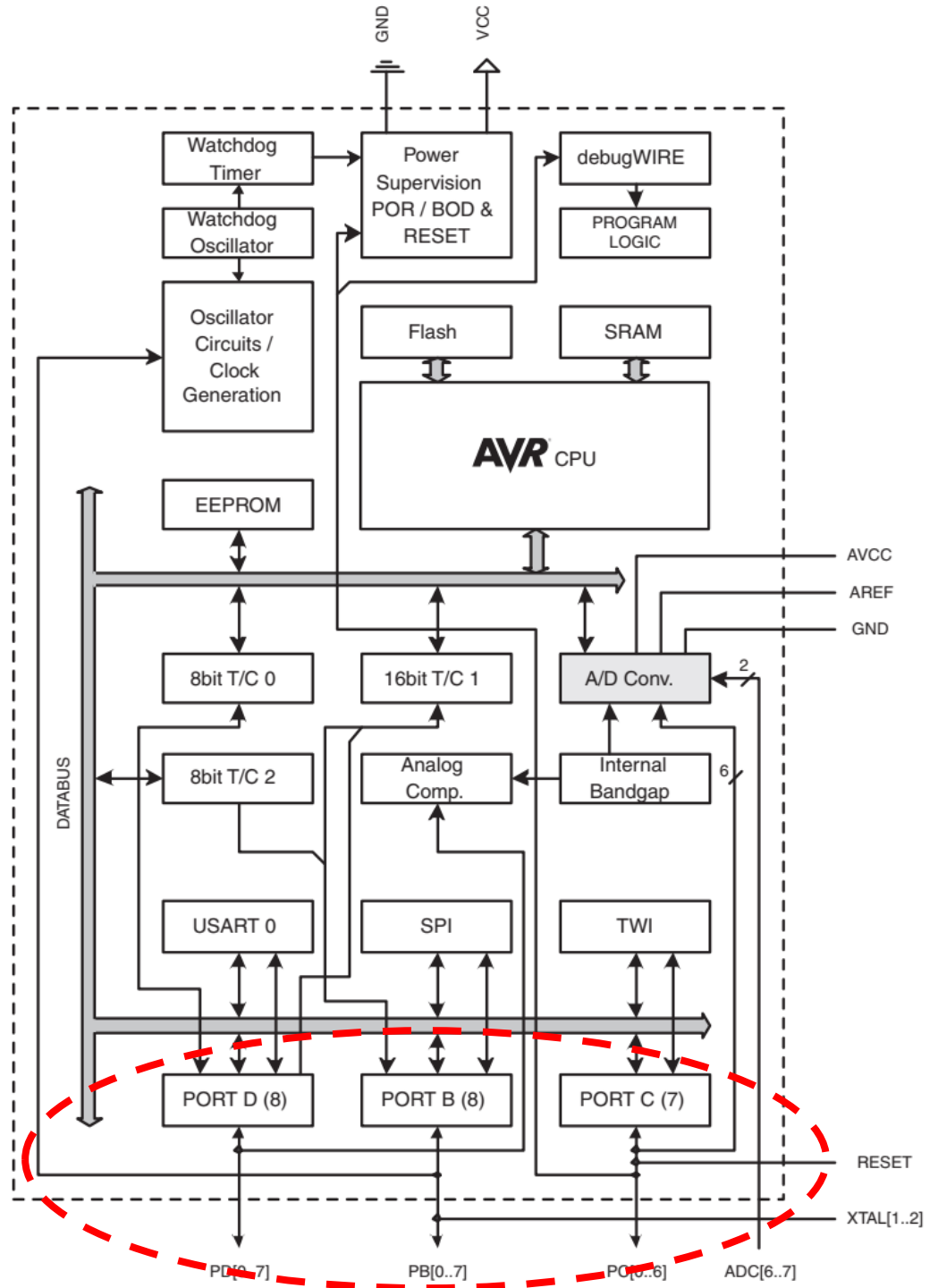
# CIRCUITOS DIGITALES Y MICROCONTROLADORES 2026

Facultad de Ingeniería  
UNLP

Programación de puertos de  
Entrada/Salida

TP1

# AVR - ATMEGA 328P



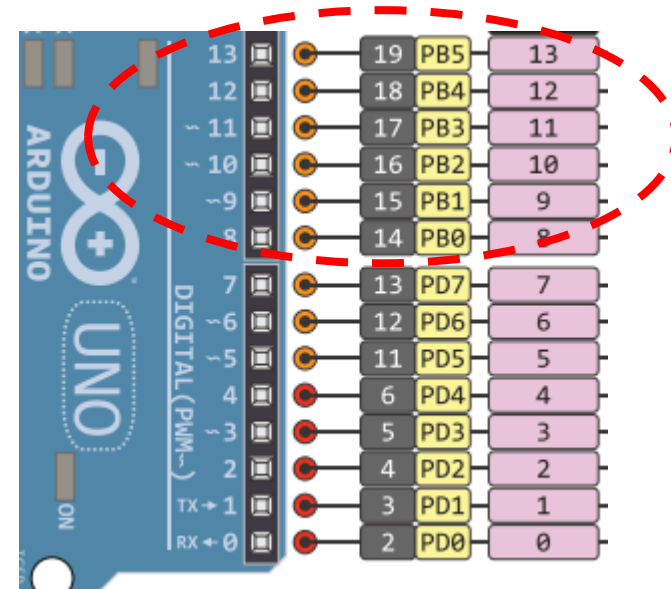
# ATMEGA - AVR

¿Donde están los terminales I/O del puerto B?

Pinout ATmega48PA/88PA/168PA/328P

|                          |    |    |                        |
|--------------------------|----|----|------------------------|
| (PCINT14/RESET) PC6      | 1  | 28 | PC5 (ADC5/SCL/PCINT13) |
| (PCINT16/RXD) PD0        | 2  | 27 | PC4 (ADC4/SDA/PCINT12) |
| (PCINT17/TXD) PD1        | 3  | 26 | PC3 (ADC3/PCINT11)     |
| (PCINT18/INT0) PD2       | 4  | 25 | PC2 (ADC2/PCINT10)     |
| (PCINT19/OC2B/INT1) PD3  | 5  | 24 | PC1 (ADC1/PCINT9)      |
| (PCINT20/XCK/T0) PD4     | 6  | 23 | PC0 (ADC0/PCINT8)      |
| VCC                      | 7  | 22 | GND                    |
| GND                      | 8  | 21 | AREF                   |
| (PCINT6/XTAL1/TOSC1) PB6 | 9  | 20 | AVCC                   |
| (PCINT7/XTAL2/TOSC2) PB7 | 10 | 19 | PB5 (SCK/PCINT5)       |
| (PCINT21/OC0B/T1) PD5    | 11 | 18 | PB4 (MISO/PCINT4)      |
| (PCINT22/OC0A/AIN0) PD6  | 12 | 17 | PB3 (MOSI/OC2A/PCINT3) |
| (PCINT23/AIN1) PD7       | 13 | 16 | PB2 (SS/OC1B/PCINT2)   |
| (PCINT0/CLKO/ICR1) PB0   | 14 | 15 | PB1 (OC1A/PCINT1)      |

No confundir el número de pin de Arduino con el número de terminal en el chip



# Registros de los Puertos I/O

¿Donde están los registros I/O del puerto B ?

descripción



## 14.4.2 PORTB – The Port B Data Register

| Bit           | 7             | 6             | 5             | 4             | 3             | 2             | 1             | 0             |              |
|---------------|---------------|---------------|---------------|---------------|---------------|---------------|---------------|---------------|--------------|
| 0x05 (0x25)   | <b>PORTB7</b> | <b>PORTB6</b> | <b>PORTB5</b> | <b>PORTB4</b> | <b>PORTB3</b> | <b>PORTB2</b> | <b>PORTB1</b> | <b>PORTB0</b> | <b>PORTB</b> |
| Read/Write    | R/W           | R/W           | R/W           | R/W           | R/W           | R/W           | R/W           | R/W           |              |
| Initial Value | 0             | 0             | 0             | 0             | 0             | 0             | 0             | 0             |              |

## 14.4.3 DDRB – The Port B Data Direction Register

| Bit           | 7           | 6           | 5           | 4           | 3           | 2           | 1           | 0           |             |
|---------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|
| 0x04 (0x24)   | <b>DDB7</b> | <b>DDB6</b> | <b>DDB5</b> | <b>DDB4</b> | <b>DDB3</b> | <b>DDB2</b> | <b>DDB1</b> | <b>DDB0</b> | <b>DDRB</b> |
| Read/Write    | R/W         | R/W         | R/W         | R/W         | R/W         | R/W         | R/W         | R/W         |             |
| Initial Value | 0           | 0           | 0           | 0           | 0           | 0           | 0           | 0           |             |

## 14.4.4 PINB – The Port B Input Pins Address<sup>(1)</sup>

| Bit           | 7            | 6            | 5            | 4            | 3            | 2            | 1            | 0            |             |
|---------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|-------------|
| 0x03 (0x23)   | <b>PINB7</b> | <b>PINB6</b> | <b>PINB5</b> | <b>PINB4</b> | <b>PINB3</b> | <b>PINB2</b> | <b>PINB1</b> | <b>PINB0</b> | <b>PINB</b> |
| Read/Write    | R/W          | R/W          | R/W          | R/W          | R/W          | R/W          | R/W          | R/W          |             |
| Initial Value | N/A          | N/A          | N/A          | N/A          | N/A          | N/A          | N/A          | N/A          |             |

# Registros de los Puertos I/O

- Como se configura la dirección

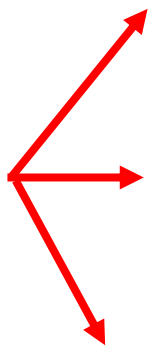
| DDxn | PORTxn | I/O    | Pull-up | Comment                                     |
|------|--------|--------|---------|---------------------------------------------|
| 0    | 0      | Input  | No      | Tri-state (Hi-Z)                            |
| 0    | 1      | Input  | Yes     | Pxn will source current if ext. pulled low. |
| 1    | 0      | Output | No      | Output Low (Sink)                           |
| 1    | 1      | Output | No      | Output High (Source)                        |

Veremos en otra clase la configuración de entrada con pull up y la estructura interna de los puertos I/O

# Programando PORTS en C

¿Cómo se utilizan los registros en C?

variables uint8 que llevan  
los mismos nombres que  
los registros



```
DDRB= 0b01010110;
```

```
PORTB= 0xC3;
```

```
mi_var = PINB; //leer el registro de entrada y  
copiarlo a una variable
```

La asociación entre las variables y los registros se realiza en el  
archivo de cabecera `io.h`

# Programando PORTS en C

Ejemplo: escribir un valor 0xAA al PORTD

```
#include <avr/io.h>

int main ()
{
    DDRD = 0xFF;           DDRD=11111111
    PORTD = 0xAA;          PORTD=10101010

    while (1);             Loop
    return 0;
}
```

# Programando PORTS en C

## Ejemplo: Leer, modificar y escribir un PORT

```
#include <avr/io.h>
```

← Incluye cabecera

```
int main ()
```

← Función principal

```
{
```

```
    DDRB = 0xFF;
```

← Inicialización

```
    while (1){
```

← Bucle infinito

```
        PORTD = PORTD+1;
```

← Operaciones: Leer-sumar-escribir

```
    }
```

```
    return 0;
```

← Fin función principal  
(nunca se alcanza)

```
}
```



# Programando PORTS en C

Ejemplo: transferir entradas a salidas

```
#include <avr/io.h>

int main ()
{
    DDRB = 0x00;
    DDRC = 0xFF;

    while (1){
        PORTC = PINB;
    }
    return 0;
}
```

---

# Programando PORTS en C

Ejemplo: obtener PINB + PINC y escribir el resultado en PORTD.

```
#include <avr/io.h>
```

```
int main ()
```

```
{
```

```
    DDRB = 0x00;
```

Todos los bits del PORTB como entradas

```
    DDRC = 0x00;
```

Todos los bits del PORTC como entradas

```
    DDRD = 0xFF;
```

Todos los bits del PORTD como salidas

```
    while (1)
```

```
        PORTD = PINB + PINC;
```

```
        return 0;
```

```
}
```

# Programando PORTS en C

Ejemplo: visualizar un contador de 8 bits en un PORT

```
#include <avr/io.h>

int main ()
{
    unsigned char z;
    DDRB = 0xFF;
    for(z=0; z<256; z++){
        PORTB=z;
    }
    while (1);
    return 0;
}
```

Resultado:

PORTB = 0x00, 0x01, 0x02, ... , 0xFF

# Programando PORTS en C

Ejemplo: escribir caracteres ASCII en el PORTB

```
#include <avr/io.h>
int main ()
{
    unsigned char myList[]="012345ABCD";
    unsigned char z;
    DDRB = 0xFF;
    for(z=0;z<10;z++){
        PORTB=myList[z];
    }
    while (1);
    return 0;
}
```

¿qué representan los bits en el PORTB?

# Programando PORTS en C

## Ejemplo: Trabajando con números con signo (ca2)

```
#include <avr/io.h>

int main ()
{
    char num[]={-4,-3,-2,-1,0,1,2,3,4};
    unsigned char z;
    DDRB = 0xFF;
    for(z=0;z<9;z++){
        PORTB=num[z];
    }
    while (1);
    return 0;
}
```

Resultado:

PORTB = 0xFC, 0xFD, 0xFE, 0xFF, 0x00, 0x01, 0x02, 0x03, 0x04

# Programando PORTS en C

¿Cómo accedemos a un solo terminal I/O del puerto B ?

¿Cómo modificamos 1bit sin alterar el resto del contenido?

## 14.4.2 PORTB – The Port B Data Register

|               |        |        |        |        |        |        |        |        |       |
|---------------|--------|--------|--------|--------|--------|--------|--------|--------|-------|
| Bit           | 7      | 6      | 5      | 4      | 3      | 2      | 1      | 0      |       |
| 0x05 (0x25)   | PORTB7 | PORTB6 | PORTB5 | PORTB4 | PORTB3 | PORTB2 | PORTB1 | PORTB0 | PORTB |
| Read/Write    | R/W    | R/W    | R/W    | R/W    | R/W    | R/W    | R/W    | R/W    |       |
| Initial Value | 0      | 0      | 0      | 0      | 0      | 0      | 0      | 0      |       |

## 14.4.3 DDRB – The Port B Data Direction Register

|               |      |      |      |      |      |      |      |      |      |
|---------------|------|------|------|------|------|------|------|------|------|
| Bit           | 7    | 6    | 5    | 4    | 3    | 2    | 1    | 0    |      |
| 0x04 (0x24)   | DDB7 | DDB6 | DDB5 | DDB4 | DDB3 | DDB2 | DDB1 | DDB0 | DDRB |
| Read/Write    | R/W  | R/W  | R/W  | R/W  | R/W  | R/W  | R/W  | R/W  |      |
| Initial Value | 0    | 0    | 0    | 0    | 0    | 0    | 0    | 0    |      |

Terminal  
PB0

## 14.4.4 PINB – The Port B Input Pins Address<sup>(1)</sup>

|               |       |       |       |       |       |       |       |       |      |
|---------------|-------|-------|-------|-------|-------|-------|-------|-------|------|
| Bit           | 7     | 6     | 5     | 4     | 3     | 2     | 1     | 0     |      |
| 0x03 (0x23)   | PINB7 | PINB6 | PINB5 | PINB4 | PINB3 | PINB2 | PINB1 | PINB0 | PINB |
| Read/Write    | R/W   | R/W   | R/W   | R/W   | R/W   | R/W   | R/W   | R/W   |      |
| Initial Value | N/A   | N/A   | N/A   | N/A   | N/A   | N/A   | N/A   | N/A   |      |

# Operadores lógicos bit a bit

Table 7-3: Bit-wise Logic Operators for C

|   |   | AND | OR  | EX-OR | Inverter |
|---|---|-----|-----|-------|----------|
| A | B | A&B | A B | A^B   | Y=~B     |
| 0 | 0 | 0   | 0   | 0     | 1        |
| 0 | 1 | 0   | 1   | 1     | 0        |
| 1 | 0 | 0   | 1   | 1     |          |
| 1 | 1 | 1   | 1   | 0     |          |

AND

```
1110 1111
& 0000 0001
-----
0000 0001
```

OR

```
1110 1111
| 0000 0001
-----
1110 1111
```

NOT

```
~ 1110 1011
-----
0001 0100
```

# Operadores de desplazamiento

data >> number of bits to be shifted right

data << number of bits to be shifted left

1110 0000 >> 3  
-----  
0001 1100

0000 0001 <<2  
-----  
0000 0100



# Operaciones Lógicas con bits

Ejemplo: Trabajando con operadores lógicos a nivel de bits

```
#include <avr/io.h>

int main ()
{
    DDRB = 0xFF;

    while (1){

        PORTB = PORTB | 0b00010000;
        PORTB = PORTB & 0b11101111;
    }
    return 0;
}
```

# Operaciones Lógicas con bits

## Ejemplo: Trabajando con operadores lógicos a nivel de bits

```
#include <avr/io.h>
```

```
int main ()
```

```
{
```

```
    DDRB =0xFF;
```

```
    DDRC = 0x00;
```

```
    while (1){
```

```
        if(PINC & 0b00100000){
```

```
            PORTB=0x55;
```

```
        }
```

```
        else{
```

```
            PORTB=0xAA;
```

```
        }
```

```
    }
```

```
    return 0;
```

```
}
```

```
PINC  XXXX XXXX  
& 0010 0000
```

-----

```
00x0 0000
```

¿resultados  
posibles?

# Simplificaciones Lógicas con I/O

Establecer un 1 lógico en el bit 4 del PORTB

```
XXXX XXXX
| 0001 0000
-----
xxx1 xxxx
```

Otra forma de escribir la mascara:

```
XXXX XXXX
| 1 << 4
-----
xxx1 xxxx
```

```
PORTB |= (1<<4); //set bit 4 (5th bit) of PORTB
```

Notar que de esta manera no se modifican los otros bits del mismo registro

# Simplificaciones Lógicas con I/O

Establecer un 0 lógico en el bit 4 del PORTB

```
      XXXX  XXXX
&    1110 1111
-----
      xxx0  xxxx
```

Otra forma de escribir la mascara:

```
      XXXX  XXXX
&    ~(1 << 4)
-----
      xxx0  xxxx
```

```
PORTB &= ~(1<<4); //clear bit 4 (5th bit) of PORTB
```

# Simplificaciones Lógicas con I/O

¿está el bit 5 de PINC en 1?

PINC    XXXX XXXX  
&       0010 0000 == (1 << 5)  
-----  
         00x0 0000  
         x puede ser 0 ó 1

```
if( ((PINC & (1<<5)) != 0)    //check bit 5 (6th bit)
```

o directamente:

```
if(((PINC & (1<<5)))    //check bit 5 (6th bit)
```

# Operaciones sobre registros I/O

- Resumiendo

```
X |= (1 << Bitnumber);    // setear un bit de la variable x, SIN AFECTAR EL RESTO
X &= ~(1 << Bitnumber);   // borrar un bit de la variable x, SIN AFECTAR EL RESTO
```

- Ejemplo:

```
#include <avr/io.h>
...
PORTD |= (1 << 2);        //setear bit 2 del port D
PORTD &= ~(1 << 2);       //borrar bit 2 del port D
```

- Utilizando los macros definidos en <avr/io.h>

```
#include <avr/io.h>
...
DDRD &= ~( (1<<PORTD0) | (1<<PORTD3) );    /* Setear PD0 y PD3 como inputs */
PORTD |= (1<<PORTD0) | (1<<PORTD3);        /* Habilitar sus Pull UP s internos */
```

# Utilizando AVR LIBC

- Un retardo de tiempo puede implementarse simplemente así:

...pero es dependiente del compilador y las optimizaciones y es difícil de ajustar

```
void delay(void)
{
    unsigned short i;
    for(i = 0; i < 42150; i++)
    { ; }
}
```

- Biblioteca de retardos:
  - Se debe especificar ANTES la constante F\_CPU con la frecuencia de reloj [MHz]

```
#define F_CPU 16000000UL
#include <util/delay.h>
```

```
    _delay_us(200);    //200 microseconds
    _delay_ms(100);    //100 milliseconds
```

- Se implementa directamente en assembler